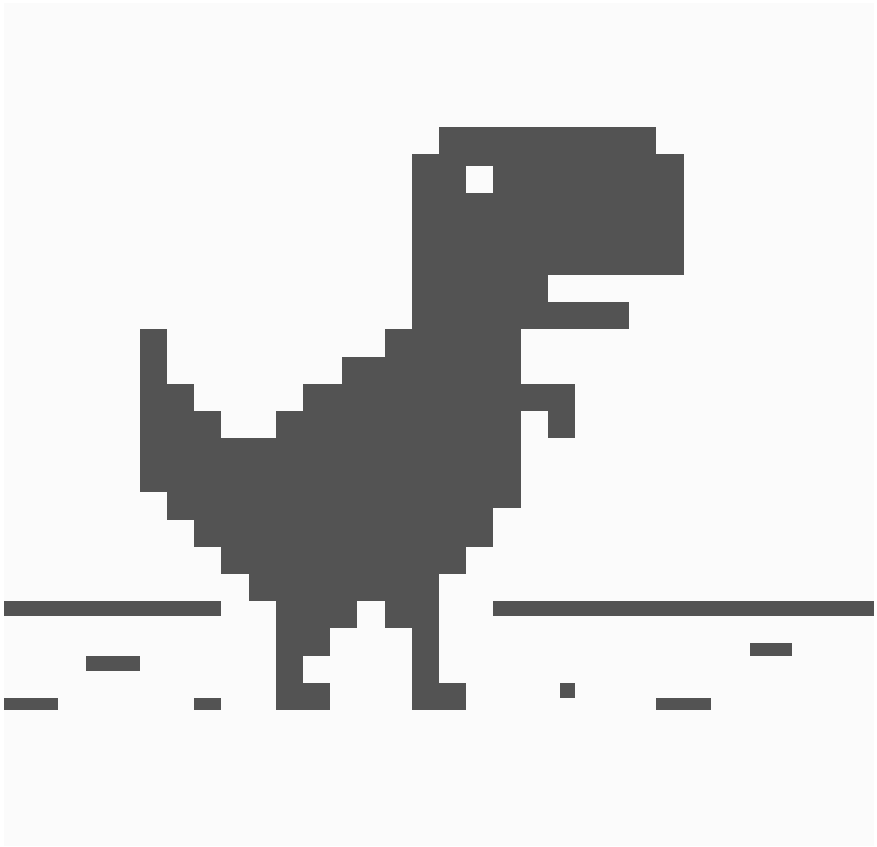
Lab Manual



ROARS, George Mason University

This manual, inspired by Mehr's 2020 biologist manual, serves as a guide for prospective and current members of Roars Lab. It outlines the lab's policies, procedures, and expectations to ensure a productive and collaborative research environment.

***Updating the manual**: This document is a living document in Roars Lab's Github repo at*

https://code.roars.dev/phd-cs-us/lab-manual.typ

Lab members, who all have write access, are encouraged to provide feedback and suggest changes to improve its clarity and usefulness.

Contents

1. ROARS Lab History and Members 4
   - 1.1. History 4
   - 1.2. Members 4
     - 1.2.1. Current members 4
     - 1.2.2. Alumni 4
2. Meetings 4
   - 2.1. Weekly lab meeting 4
3. Publication 4
   - 3.1. Authorship policy 4
   - 3.2. Venue selection 5
4. Writing Styles and File Organization 5
   - 4.1. Writing Style 5
   - 4.2. File Organization 6
   - 4.3. Writing Order for Research Papers 7
   - 4.4. Sending Me Drafts 8
   - 4.5. Rebuttal 9
   - 4.6. Presentation 9
5. Research Practice 9
   - 5.1. Data and Code Management 9
   - 5.2. Acting on Opportunities 9
6. Lab Resources 9
   - 6.1. Web Links 9
   - 6.2. Poster Printing 10
   - 6.3. Discord Server 10
   - 6.4. Computer Server Specs 10
     - 6.4.1. Getting an Account 11
     - 6.4.2. Miscellaneous 11
     - 6.4.3. Pictures 11
   - 6.5. External Resources 14
     - 6.5.1. VPN for Lab Servers 14
     - 6.5.2. Connecting to GMU VPN (VPN1) 15
     - 6.5.3. Connecting to Wifi `eduroam` 15
7. Conference Registration and Travel 16
   - 7.1. Reimbursement Process 16
     - 7.1.1. General Process 16
     - 7.1.2. Meals 16
     - 7.1.3. Car Rental 17
8. Comprehensive Exam and Defense 17

8.1. Defense 17
9. Courses and Independent Study 17
10. Contact information 17
11. LOR 18
11.1. Asking 18
11.2. Writing 18
11.3. Sending 19
11.4. Updating 19
12. Stuff for New Faculty 19
12.1. Getting Started 19

1. ROARS Lab History and Members

1.1. History

Originally founded around 2019 as the **UNSAT** (University of Nebraska Software Analysis and Testing), the lab was renamed the **ROARS** (Research on Automated Reasoning Systems) in 2021 when I moved to George Mason. The lab's research focuses on automated reasoning, software analysis, and testing, with an emphasis on developing tools and techniques that improve software reliability and security.

1.2. Members

- PI: ThanhVu (Vu) Nguyen, PhD, University of New Mexico, 2014

1.2.1. Current members

1. Long Doan, PhD student, George Mason University, since 2024
2. Hai Duong, PhD student, George Mason University, since 2022
3. Nguyen Ho, PhD student, George Mason University, since 2024
4. Linhan Li, PhD student, George Mason University, since 2022

1.2.2. Alumni

1. Azan Rasul, NSF REU undergraduate
2. Jahnvi, MS, graduated 2026
3. Didier Ishimwe, PhD, George Mason University, 2026
4. Stefania Piciorea, B.S., 2025
5. KimHao Nguyen, B.S., 2023, Jump Trading
6. Guolong Zheng, PhD, University of Nebraska-Lincoln, 2022, Minjiang University
7. Alexey Malyshev, M.S., 2021, Oracle

2. Meetings

2.1. Weekly lab meeting

Lab meeting is mandatory and normally meets in person every Thursday from 1:00 to 2:00 p.m., right before the Software Engineering seminar. The meeting is in the main Computer Science conference room, Room 4201 in Nguyen Engineering Building. If you need to attend remotely, join at meet.roars.dev and inform the PI ahead of time.

3. Publication

3.1. Authorship policy

Research papers are collaborative, and deciding who should be an author and the order of authorship can be challenging. The following guidelines can help us make those decisions:

- The **first author** is often the person who led the work and wrote most of the paper. In many cases, that is the student or other researcher who did most of the technical work and knows the approach and results best. For some papers, such as a reflection or survey of a field, the PI

may lead the writing. Papers may have multiple first authors when contributions are shared; in computer science, this is often indicated with a footnote.

- The **last author** is typically the PI or another senior researcher who supported the work through funding, advising, mentoring, discussion, ideas, experiments, strategy, or feedback on the paper and rebuttal.
- Other authors are generally ordered by their contributions. There is no strict ordering rule; when contributions are similar, authors may be listed alphabetically.
- Anyone who contributes meaningfully to the work should be included as an author. Contributions can include technical work, experiments, or writing.
- If you want to include someone whose authorship is not clear, discuss it with the PI before promising authorship or raising it with other collaborators. Discuss authorship with the potential authors early, and confirm that they want to be included.
- ACM submissions require each author to have made a substantial intellectual contribution, participated in drafting or revising the manuscript, know that it has been submitted, and agree to be accountable for issues of correctness or integrity.

3.2. Venue selection

We aim to submit work to high-quality, prestigious venues that fit the project. Strong venues offer rigorous peer review, an active research community, visibility, and the opportunity for impact. Depending on the topic, venues we consider include software engineering (ICSE, FSE, ASE, ISSTA), programming languages (POPL, PLDI, OOPSLA), logic (CAV, TACAS), and AI/ML (NeurIPS, ICML, ICLR, AAAI). Conferences are the primary publication venue in many areas of computer science; for some projects, we may submit extended versions to journals such as TSE or TOSEM.

4. Writing Styles and File Organization

Everyone has their own way of organizing files and writing papers. Below is my approach that has worked well for me over the years. If you're my student and write papers with me, I expect you follow this approach so that we can be on the same page and work efficiently together.

Note that if we collaborate with others, we can adapt how we write and organize files as needed. Typically, the main group lead will decide how to organize things and the rest of us will follow their lead. For example, if we lead a paper, then we will follow my approach below. But if we collaborate on a paper led by others, then we will follow their approach.

4.1. Writing Style

- Tell a clear story and use concise language. Prefer direct sentences and avoid passive voice when it makes the writing less clear.
- Keep paragraphs focused and reasonably short. Start a new paragraph for a new idea, and keep the sentences in each paragraph connected.
- Use “such as” for examples and “like” for similarity. Spell out numbers below ten in prose.
- The introduction should interest readers and give them a high-level understanding of the paper's main ideas and takeaways.

4.2. File Organization

1. *Use Git control (e.g., Github) for version control.* This is essential for managing changes to the paper, especially when collaborating with others.
    - Even if we use something like `Overleaf`, link it to a Github repo.
2. *Keep it simple.* I prefer a very simple file organization structure for writing papers. As few files and directories as possible.
    1. `paper.tex`: a *single* `TeX` file for the entire paper.
        - Some might prefer using multiple files / directories (e.g., `intro.tex`, `eval.tex`, `sections/related.tex`). But I find it easier to just use a single file because I can easily search and replace for things and I don't have to switch between files, which interrupts my flow.
        - Even when sharing or collaborating with others, where conflict edits can arise, a single file still works well because Git is pretty good at resolving conflict issues.
        - I use this approach for all projects, including proposal (`proposal.tex`) and presentation (`slides.tex`) files.
        - Sometimes I append things to the end of the file to indicate the venue (e.g., `paper-cacm26.tex` for a paper submitted to CACM). But the main idea still remains: an entire paper in a single file.
    2. `paper.bib`: A single `bib` file for bibliography references.
        - Using a *single* `bib` file makes it easier to manage and search for references. Others might prefer multiple bib files (e.g., `refs1.bib`, `refs2.bib` for different sections).
        - My collaborators sometimes put in their own bib files.
        - Do not symbolically link to a shared bib file (e.g., `~/myrefs.bib`) because it can cause issues when sharing the paper with others (e.g., for review or collaboration), and also if you update the shared bib file, it can cause issues for other work that also uses that bib file. Instead, just copy the bib file to the paper directory and use it there and make things self-contained.
        - `bibentry` conventions: use `authorYearFirstWordOfTitle` as the bib entry key, e.g., `duong2020verifying` for a paper by Duong in 2020 with the title "Verifying ...". This makes it easier to remember and use the bib entries in the paper. Note that Google Scholar also uses this convention when you export bib files, so it is easy to copy and paste from Google Scholar to your bib file.
            - If there are multiple papers by the same author in the same year, add a number to the end of the bib entry key, e.g., `duong2026verifying`, `duong2026verifying2`, etc.
    3. `figures/`: A directory (folder) containing all figures for the paper.
        - If there are just a few figures, I might just put them in the same directory as the paper file. But if there are many figures, I prefer to keep them in a separate directory.
        - Same as the bib file, do not symbolically link to a shared figures directory (e.g., `~/myfigs/`) because it can cause issues when sharing the paper with others.

4.3. Writing Order for Research Papers

I often write papers in the following order. For a concrete example, see the `Dynaplex` paper published at `OOPSLA'21`.

1. **Overview** section (Section 2).
    - Start with this section, which should contain an illustrative or concrete example.
    - After reading this illustrative example (and perhaps the **Introduction**), the readers should have a good understanding of the approach.
    - *The goal is to have the readers (or reviewers) make up their mind whether they like or do not like the proposed work after reading this part.*
    - A good place to add some overview figure highlighting the architecture and main components of the technique or tool.
    - Can also summarize background or important concepts in this section.
    - Typically contains:
        - An architecture or overview figure of the approach (e.g., Fig 1 in the `Dynaplex` paper)
        - A motivation or illustration example
            - Sec 2.1: Show how `Dynaplex` works at high level on an input program, what results does it produce.
        - Main definitions and background (optional)
            - Sec 2.2: General background on dynamic inference and specific features/properties of `Dynaplex`

- **Experimental** section (Section 4).
    1. Some implementation details. E.g., how the technique is implemented, what tools or libraries are used, etc.
    2. Research Questions (RQs). E.g., RQ1: How accurate is the technique? RQ2: How efficient is the technique? RQ3: How does it compare to existing approaches?
    3. Benchmark programs: describe benchmarks used to answer the RQs, and why they are good benchmarks for this purpose.
    4. Experiment Setup
        - Machine specs
        - How experiments were run (multiple times and report the mean, etc.)
    5. Results and Analysis
        - For each RQ, present the results and analyze them. E.g., for RQ1, show the accuracy results and analyze them (e.g., why is it accurate or not accurate? What are the factors that affect accuracy? etc.)
            - Don't just focus on positive results. If there are negative results, report and analyze them as well. E.g., for benchmark X, the technique does not perform well because of reasons Y and Z.

- **Algorithm/Techniques** (Section 3)

- **Related Work** (Section 5)
  - Break into multiple subsections if necessary
  - After each subsection that talks about related work, have a new paragraph and talk about why my work is different/related to other work.
- **Conclusion** (Section 6).
  - This should be the *conclusion* not the *summary*.
  - Future work can also go here
- **Introduction** (Section 1).
  - Write this last, because by the time I write the Intro, I would already have all needed info (e.g., so I can summarize the diff with state of the art, technical contributions, and results).
  - Typically contains several main things:
    - What is the problem? Why is it important (i.e., if we can solve it, then what do we accomplish)?
    - Existing work: basically answering why it is challenging (i.e., why existing work is not sufficient)?
    - The proposed approach of this paper: how does it address the limitations of existing work listed above? What other benefits does it have?
      - Results: briefly summarize the results.

4.4. Sending Me Drafts

When you have a draft ready, share it with me and I'll provide feedback. Several things to keep in mind when sharing drafts with me:

1. Use **Github** for sharing drafts. If you use Overleaf then make sure you link it to a Github repo that I have write access to. This is because I often make edits off-line and using an external editor (e.g., `VSCode` or `Emacs`).
2. **Do not** have too many `TODO` or `FIXME` or `CITE` in the draft. I don't need everything to be perfect but when there are just too many of these, it greatly disrupts my reading flow and makes it hard for me to understand the paper.
3. Do not send me very incomplete drafts. If there are incomplete sections, I will likely just skip them and not read them. If you don't have a complete draft, but some sections are complete, then you can share those sections with me and I can provide feedback on those sections.
4. Make sure your draft has *complete* **Overview** and **Experimental** sections. **Overview**, which includes an illustration of the technique, is the most important as it helps me---who reads as a reviewer---understand the approach and the main ideas of the paper. **Experimental** is also important because it shows the results and the analysis. This allows me to suggest improvements to the experiments and results, which can take a lot of time and effort to improve.
   - It is also helpful if you have the **Introduction** section complete as well, and if it is available then I will start with the Introduction because it gives me a good overview of the paper and helps me understand the motivation and contributions.

4.5. Rebuttal

1. **Thank you note**: Start with a thank you note to the reviewers for their time and feedback. Reviewing is a thankless job and reviewers often put in lots of time and effort to provide feedback. Even if the reviews are harsh and you hate them, **do it anyway**.

4.6. Presentation

1. **Title slide** with paper title, authors (highlight or underline the presenter's name), affiliations, conference or event name, date.
2. **Number your slides**. This is useful when people want to ask questions (or give feedback) on specific slides (e.g., "on slide 10, you said ...").
3. **Every slide takes approximately 1 minute**. Using this rule of thumb, you can estimate how many slides you need for your presentation.
4. **Avoid Technical Stuff**.
    - Avoid slides that are too technical (e.g., formulae, code snippets, algorithms, etc) unless absolutely necessary.
        - *Focus on motivation and high-level ideas.* The goal of the presentation is to get people interested into your work and talk to you and read your paper, which has more technical details.
5. [Optional] Use **LaTeX/Beamer** for slides. It's easier to update and maintain consistency with the paper.

5. Research Practice

5.1. Data and Code Management

When project agreements, privacy commitments, and applicable policies allow, make research data, prototypes, tools, and results available in the project's GitHub repository. Include a `README.md` describing the project and explaining how to install and use its tools. For conference papers with experiments, include scripts to generate the data and run the experiments.

5.2. Acting on Opportunities

Research opportunities can appear quickly after quiet periods, including new ideas, collaborations, funding sources, or submission venues. Be ready to act when a suitable opportunity arises. If you see something relevant to the group, share it in the research Discord or email the PI and lab members.

6. Lab Resources

6.1. Web Links

- **Website**: roars.dev. This is the main place for lab news, announcements, etc.
- **Zoom** for meetings meet.roars.dev: general lab meetings.

6.2. Poster Printing

George Mason University Libraries provide free research poster printing for students, faculty, and staff. Submit a request at least three business days before pickup through the poster request form. See the printing policy for current requirements.

6.3. Discord Server

1. `#stats` : our most active channel, where we discuss research ideas, projects, papers, and everything else related to research and the lab. This is the main channel for research discussions and announcements.
2. `#servers` : for all questions regarding lab servers, e.g., how to get an account, how to use the servers, etc. This is the main channel for server-related discussions and announcements.
3. `#general` : where we say hello to new members

6.4. Computer Server Specs

We have *four* servers (see pictures):

1. *Boba* 🧋 : ideal for AI-based projects, e.g., generative LLMs
   - NVIDIA DGX Spark
   - CPU: NVidia GB10 Chip, 20 core Arm (10 Cortex-X925 + 10 Cortex-A725 Arm)
   - Unified Memory/Graphic: 128GB RAM
   - OS: NVIDIA DGX OS (based on Ubuntu LTS)
2. **Pizza** 🍕: ideal for projects that can leverage many cores and specialized GPU
   - Custom built
   - CPU: 32 cores, 64 threads, 3.6GHz (AMD Ryzen Threadripper PRO 5975WX)
   - GPU: 2x Nvidia RTX 4090 24GB RAM
   - Memory: 128GB RAM
   - OS: Debian Stable
3. **Sushi** 🐟: high-performance workstation
   - Lenovo ThinkStation P620
   - CPU: 64 cores, 128 threads, 4.3GHz (AMD Ryzen Threadripper PRO 3995WX)
   - GPU: Nvidia RTX 5090 24GB RAM
   - Memory: 128GB RAM
   - OS: Debian Stable
4. **Taco** 🌮: ideal for experiments that need fast CPU (but not lots of cores) and fast GPU
   - Alienware Aurora
   - CPU: 16 cores, 24 threads, 5.20GHz (Intel 12th Gen Core i9-12900KF)
   - GPU: Nvidia RTX 3080 Ti 12GB RAM
   - Memory: 64GB RAM

- OS: Debian Stable

Warning: For Debian machines, **DO NOT** install anything other than Debian stable packages to these computers (e.g., **do not mix unstable or new repositories** to `apt` sources list). You can download and install anything in your home (local) directory. If in doubt, contact us.

6.4.1. Getting an Account

To get an account on the server, send a request and your **SSH public key** (for *password-less* log in).

6.4.2. Miscellaneous

1. You can use an IDE such as `VSCode` to do remote development on these servers (just connect to the server via the builtin remote login method in VSCode). You can also use other editors that support remote development (e.g., `Emacs` + `TRAMP`).
2. Do take advantage of the multicores these machines have. Most programming languages support multiprocessing (e.g., `Python`) and can significantly reduce program runtime.
3. Periodically check `top` and `htop` to make sure **you** are not taking too much CPU or memory.
4. Periodically check `/var/tmp` or `/tmp/` to make sure **you** are not taking too much space there.

6.4.3. Pictures



Figure 1: 🐟 **Sushi**



Figure 2: 🐟 **Sushi Cooling System**



Figure 3: 🍕 **Pizza**



Figure 4: 🧋 **Boba**

6.5. External Resources

6.5.1. VPN for Lab Servers

You will need VPN and Computing access to log into lab servers remotely. Send an email to cechelp@gmu.edu and also CC the PI.

Subject: VPN and Computing Access (Mesa AD Groups)

Dear CECHelp,

I am writing to request that my NetID (**your_user_name**) be added to the Active Directory groups listed below.

Groups requested for addition:

- `CN=VS-GRA,OU=HelpDeskManagement,OU=VolgenauSchool...`
- `CN=VS-SR-VPN,OU=HelpDeskManagement,OU=VolgenauSchool...`

I have copied my advisor/supervisor, Professor Thanhvu Nguyen, on this email should you require confirmation that this access is necessary.

Thank you for your time and assistance with this.

6.5.2. Connecting to GMU VPN (VPN1)

This sets up a VPN connection to GMU on Linux using NetworkManager and OpenConnect (GMU's VPN uses the Cisco AnyConnect protocol). This is my `VPN1` setup.

Linux Setup:

1. Install the OpenConnect plugin for NetworkManager (e.g., on Debian/Ubuntu: `sudo apt install network-manager-openconnect network-manager-openconnect-gnome` on Fedora: `sudo dnf install NetworkManager-openconnect NetworkManager-openconnect-gnome`). Log out and back in (or restart NetworkManager) afterward.
2. Go to Settings → Network → VPN, click `+`, and choose **Multi-protocol VPN client (openconnect)**.
3. Configure the following settings:
   - Name: `VPN1` (any name works)
   - VPN Protocol: `Cisco AnyConnect or openconnect`
   - Gateway: `vpn.gmu.edu`
   - Leave the rest (CA certificate, proxy, certificates, etc.) as default
   - IPv4 and IPv6: leave as `Automatic` (defaults)
4. Click **Add** to save.

Connecting:

1. Turn on the `VPN1` connection in the network menu.
2. Enter your George Mason NetID and Patriot Password when prompted, and click **Login**.
3. After hitting Login/Connect, open the **Duo** app on your phone. It will ask you to authenticate; accept the request and the VPN should connect.

6.5.3. Connecting to Wifi `eduroam`

ITS Eduroam Documentation

Linux Setup:

1. Select `eduroam` from the list of available wireless networks
2. Configure the following settings:
   - Wi-Fi security: `WPA & WPA2 Enterprise`

- Authentication: `Protected EAP (PEAP)`
- Anonymous identity: leave blank
- Domain: `gmu.edu`
- Check `No CA certificate is required`
- PEAP version: `Automatic`
- Inner Authentication: `MSCHAPv2`
- Username: Your full George Mason email (e.g., `netid@gmu.edu`)
- Password: Your Patriot Password

3. Click `Connect` to establish connection

7. Conference Registration and Travel

7.1. Reimbursement Process

7.1.1. General Process

- **PI Pay**: I can pay for your **conference registration** and **airline** in advance using the lab's **P-card**. Usually we do this together for efficiency.
- **You Pay (and get reimbursed)**: Everything else (e.g., hotel, food, local transport) you pay yourself (e.g., using your own credit card) and then submit reimbursement requests to the university.
  - Keep receipts. GMU requires receipts for expenses except meals reimbursed by per diem and transportation costs below $75 per instance.
  - Submit the reimbursement request and supporting documents through the Mason Finance Gateway within 60 days after the trip or event ends. Later submissions are subject to taxation.
  - Common expenses that are not reimbursable include excessive meal or lodging charges; personal expenses and losses; fines or tickets; locksmith and towing charges; domestic rental-car insurance; luxury vehicle rentals or upgrades; airline seat upgrades, preferred seating, or early boarding; first-class airfare or Acela rail fare; local commuting expenses; baggage-handler tips; personal travel insurance; fuel service options or rental-agency fuel/EV charging fees; Mason campus parking; purchases made with reward points or airline miles; and expenses for nonbusiness companions. This list is not exhaustive, and exceptions may apply where GMU's policy says otherwise.

7.1.2. Meals

1. **Per Diem**: We are paid per diem for meals when we travel for conferences. This means you don't need to save or submit receipts for meals. If you don't eat anything (do not do this)!, you will still get the per diem amount.
2. The per diem rates depend on the city we are visiting and are determined by the US General Services Administration (GSA) website: https://www.gsa.gov/travel/plan-book/per-diem-rates. When you submit reimbursement requests for meals, you will select the location and the system will automatically calculate the per diem amount for you.
3. The first day and the last day of travel, they will not pay the full per diem rate because you are not there for the full day.

7.1.3. Car Rental

- Before driving a rental vehicle for University business, become an authorized driver by completing the required driver acknowledgement, MVR review, and Driver Awareness Training. Training must be repeated every two years.
- For **domestic rentals**, do not purchase the rental agency's insurance. George Mason's coverage applies to authorized drivers on University business, and the added rental coverage is not reimbursable. See GMU vehicle insurance.
- For **international rentals**, GMU recommends purchasing liability and physical damage coverage from the rental agency. That insurance may be reimbursable depending on the circumstances. See GMU's international travel advisory.

8. Comprehensive Exam and Defense

8.1. Defense

Chair:

- In the beginning
  - Once all the committee members are present, chair will ask the student to step out of the room (virtually) at the beginning of the meeting.
  - Chair then give a brief summary of the student's progress in the PhD program, talk about their published and under-review papers, and answer any questions the committee may have. - Chair then call the student back and give the presentation.
- End of the meeting
  - the student briefly step out so that the committee can discuss the decision
  - Then the student can be invited and informed about the decision.

9. Courses and Independent Study

You must be an advisee of the PI before registering for independent study with the PI. Students who complete the work satisfactorily typically receive an A.

10. Contact information

1. Lab website: https://roars.dev
2. Email: info@roars.dev
3. Address (PI's office):

   ```
   George Mason University
     Nguyen Engineering Building #4430
     4400 University Drive
     Fairfax, VA 22030
   ```

4. Address (package delivery, e.g., from Amazon)

```
4511 Patriot CIR
  Nguyen Engineering STE 4300
  Fairfax, VA 22030-4469
  United States
```

11. LOR

11.1. Asking

- If you want a letter from me, you should **ask** at least 2 weeks in advance (ideally a month).
  - **DO NOT** put my name in the application system without explicitly asking me first.
- If I don't know you well enough, I will let you know and advise you to find someone else. But if you insist, I will still write you a letter.
- You need to **waive your right** to view the letter. It is a confidential evaluation, and I will not write one if you insist on seeing what I write.
- You should provide me your CV and, if you'd like, your SOP, so I can determine whether I could complement some of the things you've said.
  - Feel free to provide me with any other information that you think is relevant, e.g., your grades, projects, research experience, awards, etc. I might not use them if I don't think they are relevant, but they might help if I miss something.

11.2. Writing

- I **will not ask you to draft a letter** for me. My reputation is on the line, and I do not want a student (or AI) writing a letter on my behalf.
- I **do not write negative letters**, i.e., I don't say bad things about you (even if you're bad). However, if I don't know you well then a neutral, short, or generic letter will not help your case given the competitiveness.
- It takes me about **an hour** for a strong letter for someone I know well, and about 10 minutes for a letter for someone I don't know well.
- In all letters, at the end I will include a short paragraph about **an area you need to improve**, regardless of whether you're super strong or weak. But I will phrase it in such a way that the graduate study environment or a good advisor can help you overcome it. It makes the letter more complete and not just full of praise.
- I **do not customize** the letter for specific schools.
  - You customize your SOP to explain why you want to apply to school `X` and work with professor `Y`. I do not need to explain why you want to go to school `X` or why you would be a good fit for professor `Y`. Note that if I know you and `Y` very well, then I might send a letter directly to `Y` to mention you.
- My letter will have the university logo and my (digital) signature.
  - However, I should note that when I read a LoR, I do not pay attention to whether it has a logo or signature.

11.3. Sending

- After you put my email into the application system, it will send me a request email with a unique URL to go to. It will also give me some deadline that is *likely* not the same as yours---usually later.
    - In some cases the systems don't even give me a deadline (various LoR request examples are shown in my `Demystify` book, e.g., Section 3.2.3: [[https://github.com/dynaroars/phd-cs-us/]]).
- I **do not mind if the student is anxious and sends me multiple reminders**. I will likely not respond to them. But I am not bothered by reminders. Over-communication is better than under.
- Most systems simply ask me to upload the letter---though a few have short questions like comparing the applicant to undergrads or grad students. As an adcom reviewer, I don't really pay attention to these comparisons---only the letter matters.
- I usually send my letter (in PDF) in **batch mode**, e.g., I just sent out 10+ letters all at once and cleared my inbox of these requests.
    - For each request, it took me less than 30 seconds to upload the PDF and hit `submit`. So do not worry that it is time-consuming or a burden; it's not.
- After I send my batch, I'll let you know I just sent everything and ask if I missed any.
    - I would appreciate if you share an online spreadsheet showing what schools you have asked me to write letters for so I know what to expect (and if I miss something).

11.4. Updating

- Let me know your progress and especially your outcomes, e.g., interviews, offers, and where you eventually go to. I'd love to hear these updates from you.
- This is a common courtesy. Though I probably won't remember or expect you to do this. However, if you don't this and then years later reach out for another letter, I might remember and probably don't feel too enthusiastic about writing you another letter.

12. Stuff for New Faculty

12.1. Getting Started

- Know your George Mason ID number (G#); you will need it for many university services.
- GMU sets up your email account automatically. If you want a different username, ask whether it can be changed.
- To request a personal webpage on the Computer Science home server, contact CEC IT at cechelp@gmu.edu.

1. Upgrade to **Premium Reward** plan for free. This doesn't seem to be very well-known and many faculty don't know about it (so you're welcome!).
    - Schedule for an annual health checkup with your primary care physician and get a physical exam.
        - You will need to do this for yourself and your spouse (if you have one).
    - Do the Anthem BCBS Health Check In to get the Premium Reward plan.

- Log in to Anthem Website, go to Dashboard, and click on “Health Check In” to complete the process.
    - All adult members (e.g., you and your spouse) need to do it.
    - This will takes about 6 weeks to process and then you will get the Premium Reward plan. So do it in mid-May or so to ensure you get it by July 1st, which is the start of the new fiscal year for insurance.
- It doesn't seem to be widely advertised or well-implemented, so you should check your paycheck to see lower insurance premiums after you get the Premium Reward plan. If you don't see it, contact GMU HR (you can print out the completion record on Anthem Dashboard Health Check In page to show that you have completed the process).